Beyond Pip Counting: Interpretable Models for Catan Opening Placement

Stanford CS229 Project

Jack Lofwall
Department of Computer Science
Stanford University
jlofwall@stanford.edu

John Pumayalli
Department of Computer Science
Stanford University
johnpuma@stanford.edu

Michael Reller
Department of Computer Science
Stanford University
mreller@stanford.edu

Abstract

Opening settlement placement is one of the most important decisions in Settlers of Catan, though players often rely on simple heuristics. One such heuristic is pip-counting, which places settlements based solely on the hexes most likely to produce resources. Using 99,999 simulated four-player games, with 399,996 player-openings, we investigate whether a learned model can beat pip counting at predicting wins, and whether the model's learned signals are interpretable. Predicting a win on just pip counting (AUC 0.54) is nearly no better than picking a winner at random, while a linear model with engineered features reaches AUC 0.72. Resource scarcity and turn order are the strongest predictors, factors overlooked by pip counting. Only eight human-readable rules recover 62% of the model's edge over pip counting.

1 Introduction

Settlers of Catan is a stochastic, imperfect-information board game in which players' initial settlement placements are among the most consequential decisions affecting the game's outcome. The decision space for initial settlements is large, with hundreds to thousands of legal two-settlement combinations per board. Typical strategies rely on simple heuristics such as 'pip counting' (the summed roll probability of adjacent hexes) and resource diversity. Prior work on Catan has focused on building self-play agents that optimize win rate. We propose a different angle: (1) whether a supervised model can beat pip counting as a placement eval heuristic, and (2) whether the model's decisions can be translated into human-interpretable rules that a real player could use. This is an application project aimed at applying supervised and unsupervised learning to an under-explored interpretability angle on Catan opening placement. Specifically, the input to our algorithm is a single player's opening settlements and roads, the board layout, and turn order. We use logistic regression, ridge regression, XGBoost, an MLP, and SVMs to output the probability the player wins and a predicted final victory-point margin. We then attribute the learned models with SHAP and distill them into a shallow rule set to learn which placement signals contribute most to winning.

2 Related Work

Prior machine learning work on Catan has focused primarily on building strong agents to play the game. Research on Catan agents has explored applying Monte-Carlo Tree Search (Szita et al., 2009) and adversarial search with deep networks (Driss and Cazenave, 2022), including deep models

focusing on settlement placement specifically (Papadam and Chalkiadakis, 2025). Even Catanatron (Collazo, 2021), a strong open-source bot and the simulator we use, is a learned agent. This research has optimized playing strength, but encodes placement quality only implicitly. This research has not isolated which placement features drive winning, benchmarked a learned model against a standard pip-counting heuristic, or distilled predictions into human-usable rules. The research closest to our work comes from Guhe and Lascarides (2014), where they hand-tuned an agent and evaluated it through self-play simulations and a human-game corpus. They found that the initial placement is decisive, as an untuned agent against their three placement-improved opponents only wins $\sim$15% of the games. The direction of this research, though, is the inverse of ours. They begin from interpretable rules and use them to tune the agent to play better. In contrast, we learn a model from game outcomes and then make it interpretable, isolating the opening as a predictive target, attributing it, and testing head-to-head against pip counting. Our interpretability analysis builds on SHAP (Lundberg and Lee, 2017), which attributes a prediction to per-feature Shapley contributions. Though SHAP has been applied to games like chess, we have not found any research on its use for Catan placement.

3 Dataset and Features

Dataset Using Catanatron's strongest documented bot, 'AlphaBetaPlayer', we generated 99,900 four-player games of self-play, using official Catan spiral number placement with randomized hex and port layout. For each game, we extracted a JSON record containing: hex coordinates, resources, numbers, ports, each player's two opening settlements and roads in placement order, player turn order, game winner, and final per-player victory point totals.

Feature Engineering From our 99,999 generated games, we build a per-player feature table of 399,996 rows (4 players per game $\times$ 99,999 games) with 52 total features that we categorize into 8 groups: (1) pip score (summed dice roll probability of each settlement's hexes), (2) resource diversity (distinct adjacent resources with per-resource boolean flags), (3) port information (settlement directly on a port, BFS distance to nearest port, and matching port types to resources owned), (4) expansion potential (count and sum of pips of legal future settlement spots), (5) opponent information (min, mean, max opponent pip, resource-overlap, and per-resource opponent counts), (6) resource scarcity (each settlements share of the total pip for each resource), (7) 'engines' / resource interactions (ore+wheat for city creation, wood+brick for road building / expansion), (8) seat index / placement order (1-4, snake draft Catan rules). Labels are boolean 'won' and vp_margin = final VP (victory points) - max(opponent VPs). For all non-tree models, we standardize features to zero mean and unit variance, and the tree models use raw features. We used a train/val/test split of 70/15/15 at the game level, ensuring all four rows of a given game land in the same partition.

4 Methods

We fit five model families to both tasks (win classification and victory point margin) and compare each against a single-feature pip-only baseline (`total_pip`), plus a simple mean baseline for regression. **Logistic regression** models $P(\texttt{won}{=}1 \mid x) = \sigma(\theta^\top x)$ with $\sigma(z) = 1/(1+e^{-z})$, fit by minimizing ℓ_2-regularized cross-entropy. **Ridge** minimizes $y - X\theta_2^2 + \alpha\theta_2^2$ with $\alpha{=}1$. **XGBoost** sums the outputs of K shallow regression trees, $\hat{y} = \sum_{k=1}^{K} f_k(x)$, adding them one at a time so each new tree is fit to the residual error left by the trees before it. We limit overfitting by keeping the trees shallow (depth = 5), reducing each tree's contribution (learning rate = 0.1), and stopping early once validation performance stops improving (at most 300 trees). The **MLP** is a feedforward network with two hidden layers $(128, 64)$, ReLU activations, ℓ_2 penalty 10^{-3}, and early stopping. For **SVMs** we fit linear SVC/SVR ($C{=}1$) on the full training set (SVM margins are not probabilities, so we map them with Platt scaling, a sigmoid fit to the scores via 3-fold CV) and an RBF-kernel SVM, $k(x, x') = \exp(-\gamma x - x'^2)$ with $\gamma = \text{scale}$, $C{=}1$. Since kernel training is $O(n^2)$, we fit the RBF on a $\sim$32K-row game-level subsample and separately benchmark its scaling. For **interpretation** we compute SHAP values on the XGBoost model and then distill it by fitting shallow decision trees (depth 2–4) to `won`, treating each leaf as an if-then rule with a win probability. For **unsupervised** structure we run k-means on standardized opening features. We run a k-sweep over $k \in [2, 8]$, where silhouette and elbow are weak and favor $k{=}3$. However, because the clusters are not clean, we instead select $k{=}5$ for interpretability, as it separates resource-specific opening types that $k{=}3$ would merge.

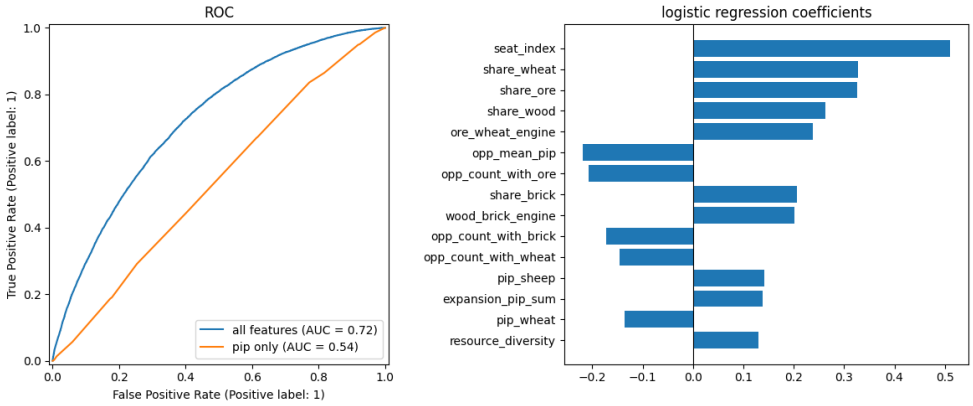

Figure 1: ROC for the full feature set vs. pip-only (left) and the largest logistic-regression coefficients (right).

Table 1: Held-out test performance. Lower Brier/RMSE and higher AUC/R^2 are better.

Model	Class AUC	Class Brier	Reg. RMSE	Reg. R^2
Pip-only baseline	0.536	0.187	3.785	0.004
Logistic / Ridge	0.721	0.166	3.440	0.178
LinearSVC / LinearSVR	0.721	0.166	3.517	0.140
SVM (RBF, $\sim$32K subsample)	0.679	0.173	3.502	0.148
MLP	0.732	0.164	3.388	0.202
XGBoost	**0.733**	**0.164**	**3.384**	**0.204**

5 Experiments / Results / Discussion

Metrics. We did not focus on raw accuracy because only $\sim$25% of openings result in a win, so an unweighted classifier can have high accuracy by guessing losses every time. Therefore, the metrics we used were AUC (Area Under the ROC Curve) and the Brier score. AUC evaluates a model on how often it can rank a winner higher than a loser, with 0.5 meaning it is random and 1.0 being perfect. The Brier score is the mean squared error of predicted probabilities; a lower score signifies a better calibrated model. We report RMSE and R^2 on the VP margin for regression.

Learned models beat pip counting. Table 1 reports test set performance. Using pip-counting to predict wins reported AUC of 0.536 (barely better than random) and R^2 of 0.004. On the other hand, the linear model with our features reached AUC 0.721 and R^2 0.178. The tree and neural models, XGBoost and MLP respectively, don't add much beyond the linear model, reaching AUC 0.733. Interestingly, the SVM model performed worse than the linear models on both tasks, suggesting the signal is largely linear. Cross validation intervals are tight ($\pm$0.002–0.003), meaning that the gains are stable and the split is not being overfit. Figure 1 confirms these results: pip counting's curve hugs the random baseline while the full model lifts off it at every threshold, and the largest model coefficients are resource shares, the ore-wheat engine, and seat order.

Which features matter most for winning? Figure 2 shows SHAP attributions for the XGBoost model. The strongest signals are seat index, the ore-wheat and wood-brick engine features, resource diversity, and opponent resource competition (ore is good, but even better if your opponents don't have access to any). We trained the model with one feature group removed at a time, leave-one-group-out ablation, to see how AUC is affected. The strongest group alone is resource scarcity (AUC 0.655), followed by seat alone (AUC 0.634) and pip alone (AUC 0.601). The most irreplaceable, meaning other features can't compensate for them, are seat ($-$0.021 AUC), opponent features ($-$0.011 AUC), and scarcity ($-$0.007 AUC).

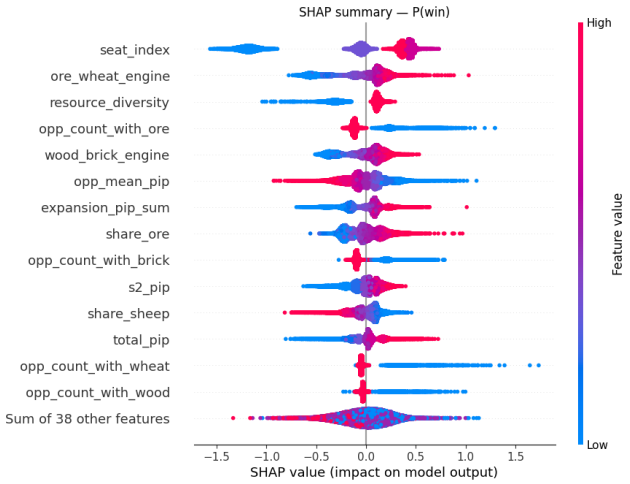

Figure 2: SHAP summary for the XGBoost win classifier; each point is a player-row, colored by feature value.

Table 2: Fraction of XGBoost's edge over pip counting recovered by a small rule set.

Predictor	Test AUC	% of edge-over-pip recovered
Pip counting	0.536	—
Rule tree (8 rules)	0.658	62%
Rule tree (16 rules)	0.680	73%
XGBoost (full)	0.733	100%

Turning the model into human-usable rules. We fit shallow decision trees to `won` and calculated how much of the AUC gap between pip counting and XGBoost they recover. Only eight rules recover 62% of the gap, and sixteen rules recover 73% (Table 2). The standout rule is that if a player is not in seat 0, and they have both an ore-wheat engine and a wood-brick engine, they win 35.9% of the time (notably above the 25% win rate). This rule applies to 46% of all openings. Most of what the model learned can be written into a manageable number of human-usable rules.

Opening archetypes (unsupervised). We ran k-means on the opening features (without using win labels) and it grouped openings into five types: ore/brick-heavy 29.4% (the only archetype above the 25% base rate), wheat 24.3%, wood 23.5%, port-focused 20%, and sheep-heavy 16.6% (Figure 3). These results agree with our supervised findings and general Catan intuition: the ore-driven engine wins, while sheep-heavy openings lose because sheep's main outlet is development cards. Development cards can lead to largest army, but it only happened in 0.1% of games, making high sheep production relatively unhelpful.

Calibration, error analysis, and overfitting. The XGBoost classifier is well-calibrated out of the box (with Brier 0.164), so Platt and isotonic recalibration change nothing. On the 5% of openings where the model and pip counting disagree most sharply, the model is far more accurate (Brier 0.237 vs. 0.348): openings that are favored by the model and not pip counting won 58.5% of the time, while the reverse produced zero wins. Pip's predictions collapse into a narrow 0.20-0.30 band that cannot discriminate. Five-fold CV intervals are tight ($\pm$0.002–0.003) and train/test behavior is matched, indicating minimal overfitting. Regression residuals are heteroscedastic: they show that opening features identify weak placements cleanly, but the magnitude of a strong outcome is largely determined by mid-game stochasticity (dice, robber) that is unknowable at placement time, making it a principled ceiling rather than a modeling failure.

Discussion. Pip counting is nearly useless alone (AUC 0.54); a linear model with our features reaches 0.72, and trees and neural nets add almost nothing (0.73). The near-zero gain from nonlinear models is itself the finding: opening quality is a largely linear, additive function of features pip

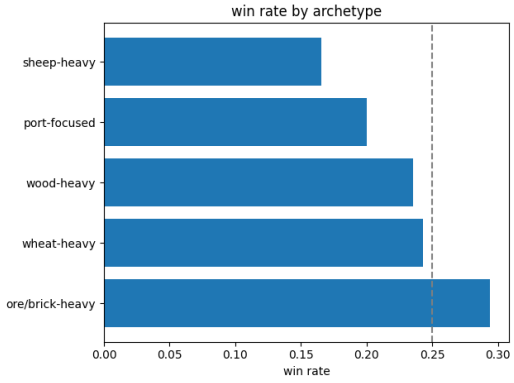

Figure 3: Win rate by unsupervised opening archetype (dashed line = 25% base rate).

counting omits, chiefly resource scarcity and turn order. The RBF kernel underperforming linear models is a clean diagnostic: the decision boundary is genuinely linear, not just approximately so. Notably, 0.73 AUC implies that roughly 27% of outcome variance is explainable from the opening alone, a surprisingly large share for a dice-heavy game and a testament to how much placement matters before a single die is rolled.

6 Conclusion / Future Work

A learned model substantially beats pip counting on both win prediction and VP margin, and the gain is interpretable: resource scarcity and turn order (not raw pip totals) drive winning, and that signal distills into a compact, human-readable rule set. The scale of the improvement is striking: pip counting, the dominant human heuristic, is nearly uninformative, and straightforward supervised learning with better features closes most of the gap to the performance ceiling set by mid-game stochasticity. With more time we would (i) generate data from multiple bot strengths to test whether these signals are agent-specific or intrinsic to Catan; (ii) extend prediction beyond the opening to early-game states; (iii) study how optimal placement varies by turn order, since seat index is the single most irreplaceable signal; (iv) test across different player counts, including the popular two-player variant, where board dynamics and resource competition differ substantially from the four-player setting; and (v) design opening-aware bots that use our placement signals rather than pip counting and re-run evaluation to measure how much stronger such agents are in practice.

7 Contributions

All three members contributed equally throughout the entire process.

References

Bryan Collazo. 2021. Catanatron: Settlers of Catan bot simulator. `https://github.com/bcollazo/catanatron`.

Brahim Driss and Tristan Cazenave. 2022. Deep Catan. In *Applications of Evolutionary Computation (EvoApplications)*.

Markus Guhe and Alex Lascarides. 2014. Game strategies for the Settlers of Catan. In *IEEE Conference on Computational Intelligence and Games (CIG)*.

Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Dimitrios R. Papadam and Georgios Chalkiadakis. 2025. Adversarial search and deep learning for strategic settlement placement in the "Settlers of Catan". In *European Conference on Multi-Agent Systems (EUMAS)*.

István Szita, Guillaume Chaslot, and Pieter Spronck. 2009. Monte-carlo tree search in Settlers of Catan. In *Advances in Computer Games (ACG)*.